

Dynamic Computation Offloading in Edge Computing for Internet of Things

Ying Chen¹, Ning Zhang², *Member, IEEE*, Yongchao Zhang, and Xin Chen

Abstract—Nowadays, billions of Internet of Things (IoT) devices arise around us running complex and computation-intensive applications. Due to the limited resources of the IoT devices, it is appealing to offload the application tasks from IoT devices to the remote cloud data centers. However, offloading all the tasks to the cloud can put a significant burden on the network. One promising way to solve this issue is edge computing, where edge servers are provisioned at the network edge. In edge computing for IoT, as the task generating process is highly dynamic and the statistical information can hardly be obtained or precisely predicted, it is of great importance yet very challenging to effectively offload application tasks to achieve the tradeoff between offloading cost and performance. In this paper, we formulate the computation offloading as an optimization problem to minimize offloading cost while providing performance guarantees. Based on stochastic optimization, we propose a dynamic computation offloading algorithm (DCOA), which decomposes the optimization problem into a series of subproblems, and solves these subproblems concurrently in an online and distributed way. Theoretical analysis is presented which demonstrates that DCOA can achieve the tradeoff between offloading cost and performance. Experiments are also carried out to evaluate the effectiveness of DCOA.

Index Terms—Computation offloading, edge computing, Internet of Things (IoT), stochastic optimization.

I. INTRODUCTION

WITH emergence of the Internet of Things (IoT), physical objects can be connected to collect and exchange information, which can greatly improve many aspects of our daily lives [1]–[4]. With the prosperity of IoT, applications running on IoT devices are becoming more and more complex and computation-intensive. However, the resources (such as computing capacity and battery) in IoT devices are generally limited. Conventionally, to solve this problem, the tasks can be offloaded to cloud data centers, which have larger computing

capacity. Nevertheless, the number of IoT devices is increasing dramatically and these IoT devices generate a large amount of data. The Cisco Global Cloud Index estimates that by 2019 the data produced by connected things, machines and people would be 500 zettabytes; however, the traffic of global data center can only reach 10.4 zettabytes [5]. Thus, offloading all the tasks from IoT devices to the cloud data center can put a significant burden on the network and cause network congestion [6].

To tackle this challenge, there is an emerging computing paradigm, which places small-scale data centers with several edge servers at the edge of the network to offer certain processing capacity. Such computing paradigm, called edge computing, has drawn extensive attention from both academy and industry [6], [7]. With the help of edge computing, application tasks running on the IoT devices can be offloaded to the edge servers in edge computing to get better service. Generally speaking, the computing resources at the edge computing system are still limited compared to remote cloud data centers. Therefore, an edge computing is customarily backed by a remote back-end cloud via the Internet [8]. Offloading tasks to the edge computing would incur less costs than offloading to the back-end cloud. However, the intuitive idea to offload all the tasks to the edge computing system cannot work well, since it may lead to too many tasks competing for the constrained resources and make the system unstable. Thus, how to effectively offload the tasks from the IoT devices to achieve the tradeoff between cost and performance is a key research issue.

It faces several challenges to address the above issue. First, the task generating process is highly dynamic and the statistical information can hardly be obtained or precisely predicted. In addition, the operating environment also varies over time. Thus, how to make dynamic offloading decisions in the face of uncertainty of task patterns is a great challenge. Besides, the number of IoT devices is increasing dramatically. These IoT devices run various kinds of applications. Consequently, the scale of the dynamic offloading problem is an enormous obstacle [9]. Existing approaches, such as combinatorial optimization or dynamic programming can suffer from high complexity, and more efficient approaches are required.

In this paper, we study the dynamic computation offloading in edge computing for IoT. An optimization model is formulated to address the research problem, where the goal is to optimize offloading cost while providing performance guarantees. Based on stochastic optimization techniques [10], we propose a dynamic computation offloading algorithm (DCOA).

Manuscript received May 19, 2018; revised July 22, 2018; accepted October 6, 2018. Date of publication October 12, 2018; date of current version June 19, 2019. This work was supported in part by the National Natural Science Foundation of China under Grant 61872044 and Grant 61502040, in part by the Key Research and Cultivation Projects at Beijing Information Science and Technology University under Grant 5211823411, and in part by the Supplementary and Supportive Project for Teachers at Beijing Information Science and Technology University under Grant 5111823401. (Corresponding authors: Ning Zhang; Xin Chen.)

Y. Chen, Y. Zhang, and X. Chen are with Computer School, Beijing Information Science and Technology University, Beijing 100101, China (e-mail: chenying@bistu.edu.cn; zhangyongchao@mail.bistu.edu.cn; chenxin@bistu.edu.cn).

N. Zhang is with Texas A&M University Corpus Christi, Corpus Christi, TX 78412 USA (e-mail: ning.zhang@tamucc.edu).

Digital Object Identifier 10.1109/JIOT.2018.2875715

DCOA can be implemented online and distributively, and it offloads the computation tasks dynamically to adapt to changing external environments. DCOA makes use of a parameter W to obtain the tradeoff between offloading cost and performance. Mathematical analysis is carried out which shows that DCOA can approach the optimal offloading cost while bounding queue length. And we conduct the experiments to evaluate the effectiveness of DCOA.

In the remaining parts of this paper, the dynamic offloading problem is formulated in Section II. To solve the dynamic offloading problem effectively, the dynamic offloading algorithm is presented in Section III. In Section IV, we carry out the mathematical analysis to elucidate the performance of DCOA. Experiments are conducted to validate DCOA in Section V. Section VI presents related work and Section VII concludes this paper.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

Consider one mobile edge computing (MEC) system which consists of n edge servers and is backed up by a back-end cloud. Each edge server j is virtualized to m virtual machines (VMs) to support m applications running on the IoT devices [11]. In particular, the i -th VM running at the j -th server processes tasks for the i -th application. Let I denote the collection of the application index i . To be general, we consider the edge servers are heterogeneous and have different processing capacities. We list the main notations in Table I.

A time-slotted model is adopted in this paper [12]. In time slot t , the number of tasks for the i -th application needing to be offloaded is denoted by $A_i(t)$. $A_i(t)$ varies over time and there may be bursty arrivals. Let $a_{ij}(t)$ represent the number of tasks for the i -th application that are offloaded to server j . We consider that the processing capacity of back-end cloud is sufficient, and it can be regarded as a large resource-pool; therefore, for brevity of symbol expression and problem formulation, we use $j = n + 1$ to denote the back-end cloud [9], [12]. Let J be the collection of index j . For any slot t and any $i \in I$, it holds that

$$\sum_{j \in J} a_{ij}(t) = A_i(t). \quad (1)$$

We study the dynamic computation offloading in the MEC for IoT. Here, $a_{ij}(t)$ is the decision variable. Note that our model requires no statistical knowledge of $A_i(t)$, which is generally hard to precisely predict or obtain in practice.

B. Cost Model

It would incur costs to offload tasks to the MEC and the back-end cloud. Let $\varphi_i(\cdot)$ denote the cost function of offloading tasks of the i -th application to the MEC. Specifically, $\varphi_i(x, t)$ represents the cost for offloading x tasks of the i -th application to the MEC during time slot t . $\varphi_i(x, t)$ can be affected by the application index i , the number of offloaded tasks and the current time slot t . It can also be related with other factors, such as the wireless fading, traffic, etc. Let $\phi_i(\cdot)$ be the cost

TABLE I
NOTATIONS AND DEFINITIONS

Notations	Definitions
I	Applications set.
J	Servers set.
$A_i(t)$	Number of offloaded tasks for the i th application in slot t .
$a_{ij}(t)$	Number of tasks for the i th application that are offloaded to server j in slot t .
$b_{ij}(t)$	The processing ability of the j th server towards the i th application in slot t .
$\varphi_i(\cdot)$	Cost function of offloading tasks for the i th application to the MEC.
$\phi_i(\cdot)$	Cost function of offloading tasks for the i th application to the back-end cloud.
$Q_{ij}(t)$	Queue length of tasks for the i th application running at server j in time slot t .
q_{ij}	Average queue length of tasks for the i th application running at server j .
$f_i(t)$	Offloading cost for the i th application during time slot t .
$g(t)$	Total offloading cost for the applications during time slot t .
g	Time average offloading cost for the applications.
W	Tradeoff parameter between the queue length and offloading cost.

function of offloading tasks of the i -th application to the back-end cloud. Similarly, $\phi_i(x, t)$ represents the cost for offloading x tasks of the i -th application to the back-end cloud in time slot t . For the functions of $\varphi_i(x, t)$ and $\phi_i(x, t)$, we can easily get the following property.

Property 1: For each application i , no matter what the slot t is, the following conditions should be satisfied for the $\varphi_i(x, t)$ and $\phi_i(x, t)$.

- 1) $\varphi_i(0, t) = 0$.
- 2) $\phi_i(0, t) = 0$.
- 3) If $x_1 < x_2$, then $\varphi_i(x_1, t) \leq \varphi_i(x_2, t)$.
- 4) If $x_1 < x_2$, then $\phi_i(x_1, t) \leq \phi_i(x_2, t)$.
- 5) For any $x > 0$, $\varphi_i(x, t) < \phi_i(x, t)$.

The first four conditions guarantee that the cost functions capture the nondecreasing feature between the costs and the number of offloaded tasks. The first two conditions are obvious since it would incur no costs if no tasks are offloaded. The last condition means that offloading tasks from the IoT devices to the back-end cloud would incur more costs than offloading to the MEC. This is because cloud is usually deployed remotely, and offloading tasks to cloud would have higher transmission costs, network costs, etc.

In slot t , the cost for offloading tasks for the i -th application is

$$f_i(t) = \varphi_i\left(\sum_{j=1}^n a_{ij}(t), t\right) + \phi_i(a_{i(n+1)}, t). \quad (2)$$

The cost for offloading tasks for all the applications can be represented by

$$g(t) = \sum_{i \in I} f_i(t). \quad (3)$$

Instead of studying the instantaneous cost, this paper focuses on the offloading cost in a long time duration. The long-term average cost can be denoted as

$$g = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{E}\{g(t)\} \quad (4)$$

which is the objective to be minimized in our computing offloading problem.

C. Queueing Model

For most of the applications, queueing delay affects user satisfaction degree. A large queueing delay can degrade the user experience. *Little's Law* shows that queueing delay is positively correlated to the queue length. Thus, this paper tries to decrease the queue backlog and make the MEC under low congestion state.

Let $b_{ij}(t)$ represent the processing ability of server j toward i -th application in slot t . That is to say, in slot t , server j can process at most $b_{ij}(t)$ tasks for i -th application offloaded from the devices. Then, we adopt $Q_{ij}(t)$ in slot t to represent the queue backlog for the i -th application on server j . Therefore, we have the following expression of queue backlog $Q_{ij}(t)$:

$$Q_{ij}(t+1) = \max[Q_{ij}(t) - b_{ij}(t), 0] + a_{ij}(t). \quad (5)$$

Let q_{ij} represent the time average queue backlog of the i -th application running at server j . To minish the average queue backlog and maintain stability, we seek to bound the average queue backlog, which is described in

$$q_{ij} = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{E}\{Q_{ij}(t)\} < \zeta, \exists \zeta \in \mathbb{R}^+. \quad (6)$$

D. Optimization Problem

There exists tradeoff between offloading cost and queue length. Offloading all the tasks to the MEC can reduce the cost; however, the MEC may become highly congested and unstable [8]. Combining cost and queue length, we formulate the computation offloading problem as

$$\text{minimize } g = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{E}\{g(t)\} \quad (7)$$

subject to constraints (1) and (6).

Solving the problem is difficult due to the following reasons. First, offline solution requires the future information (such as offloaded task arrivals and cost functions) which varies over time and is hard to obtain or precisely predict. How to make offloading decisions to adapt to dynamic environment is of great challenge. Second, as the number of devices is rising rapidly, the scale of the problem also increases. Thus, traditional optimization algorithms may suffer from high complexity, and an efficient algorithm is desirable.

III. DYNAMIC COMPUTATION OFFLOADING ALGORITHM

To solve the optimization problem efficiently, we design an online dynamic offloading algorithm which decomposes the optimization problem into a series of subproblems, and makes

offloading decisions in each time slot. In this section, since Lyapunov optimization techniques can simplify the stochastic problem into deterministic optimization problem and obtain the close-to-optimal solution, this paper would apply it to solve the optimization problem.

A. Problem Transformation

Here, we let $\Omega(t) = (Q_{ij}(t))$ define queue backlog matrix. Taking advantage of Lyapunov optimization techniques [10], [13], the Lyapunov function is defined as

$$H(\Omega(t)) = \frac{1}{2} \sum_{i \in I} \sum_{j \in J} Q_{ij}^2(t). \quad (8)$$

Specifically, the value of $H(\Omega(t))$ represents the congestion state of the system. Next, the *conditional Lyapunov drift* $\Delta(\Omega(t))$ is expressed in

$$\Delta(\Omega(t)) = \mathbf{E}\{H(\Omega(t+1)) - H(\Omega(t)) | \Omega(t)\}. \quad (9)$$

To combine the queue length and offloading cost, we define the *drift plus cost* in

$$\Delta(\Omega(t)) + W\mathbf{E}\{g(t) | \Omega(t)\}. \quad (10)$$

Here, $W > 0$ is the tradeoff parameter between the queue length and offloading cost. Setting W larger indicates that the offloading cost $g(t)$ is given more weight.

Next, Theorem 1 elucidates the upper bound of *drift plus cost*.

Theorem 1: In any slot t , with any computation offloading strategies, no matter what the possible values of $\Omega(t)$ and W are, if the number of offloadable tasks arrived, $A_i(t)$, is upper bounded by a peak value $A_i^{\max}$, then, there exists an upper bound for the *drift plus cost*, as expressed in

$$\begin{aligned} & \Delta(\Omega(t)) + W\mathbf{E}\{g(t) | \Omega(t)\} \\ & \leq C + \sum_{i \in I} \sum_{j \in J} Q_{ij}(t) \mathbf{E}\{a_{ij}(t) - b_{ij}(t) | \Omega(t)\} \\ & \quad + W \sum_{i \in I} \mathbf{E} \left\{ \varphi_i \left(\sum_{j=1}^n a_{ij}(t), t \right) + \phi_i(a_{i(n+1)}(t), t) | \Omega(t) \right\}. \end{aligned} \quad (11)$$

Here, $C = (1/2)[\sum_{i \in I} \sum_{j \in J} \hat{b}_{ij}^2 + \sum_{i \in I} (A_i^{\max})^2]$ is constant.

Proof: Squaring both sides of (5), and combining the inequality that $(\max[Q_{ij}(t) - b_{ij}(t), 0])^2 \leq (Q_{ij}(t) - b_{ij}(t))^2$, there holds

$$\begin{aligned} Q_{ij}^2(t+1) & \leq (Q_{ij}(t) - b_{ij}(t))^2 + a_{ij}^2(t) \\ & \quad + 2a_{ij}(t)\max[Q_{ij}(t) - b_{ij}(t), 0]. \end{aligned} \quad (12)$$

For each application $i \in I$, the number of tasks actually served at server j during slot t is represented by $\bar{b}_{ij}(t)$

$$\bar{b}_{ij}(t) = \begin{cases} b_{ij}(t), & b_{ij}(t) \leq Q_{ij}(t) \\ Q_{ij}(t), & \text{otherwise.} \end{cases} \quad (13)$$

We obtain $\max[Q_{ij}(t) - b_{ij}(t), 0] = Q_{ij}(t) - \bar{b}_{ij}(t)$. Then, (12) is reduced to

$$\begin{aligned} Q_{ij}^2(t+1) & \leq Q_{ij}^2(t) + b_{ij}^2(t) + a_{ij}^2(t) \\ & \quad + 2Q_{ij}(t)(a_{ij}(t) - b_{ij}(t)) - 2a_{ij}(t)\bar{b}_{ij}(t). \end{aligned} \quad (14)$$

Since $-2a_{ij}(t)\bar{b}_{ij}(t) \leq 0$ holds, then

$$\frac{1}{2} \left[Q_{ij}^2(t+1) - Q_{ij}^2(t) \right] \leq \frac{1}{2} \left[b_{ij}^2(t) + a_{ij}^2(t) \right] + Q_{ij}(t) [a_{ij}(t) - b_{ij}(t)]. \quad (15)$$

Next, summing up (15) over all the applications and all the servers, then, taking conditional expectations, it yields

$$\Delta(\Omega(t)) \leq \frac{1}{2} \sum_{i \in I} \sum_{j \in J} \mathbf{E} \left\{ b_{ij}^2(t) + a_{ij}^2(t) | \Omega(t) \right\} + \sum_{i \in I} \sum_{j \in J} Q_{ij}(t) \mathbf{E} \{ a_{ij}(t) - b_{ij}(t) | \Omega(t) \}. \quad (16)$$

In addition

$$\sum_{j \in J} \mathbf{E} \{ a_{ij}^2(t) | \Omega(t) \} \leq \mathbf{E} \{ A_i^2(t) | \Omega(t) \} \leq (A_i^{\max})^2. \quad (17)$$

Define $\hat{b}_{ij}$ as the upper bound of $b_{ij}(t)$ for all the time slots, then

$$\mathbf{E} \{ b_{ij}^2(t) | \Omega(t) \} < \hat{b}_{ij}^2. \quad (18)$$

Combing (17) and (18), it holds that

$$\sum_{i \in I} \sum_{j \in J} \mathbf{E} \left\{ b_{ij}^2(t) + a_{ij}^2(t) | \Omega(t) \right\} \leq \sum_{i \in I} \left\{ \sum_{j \in J} \hat{b}_{ij}^2 + (A_i^{\max})^2 \right\}.$$

By adding $\mathbf{WE} \{ g(t) | \Omega(t) \}$ to the both sides of the above inequation, and then, making C equal to $(1/2) [\sum_{i \in I} \sum_{j \in J} \hat{b}_{ij}^2 + \sum_{i \in I} (A_i^{\max})^2]$, we get

$$\begin{aligned} \Delta(\Omega(t)) + \mathbf{WE} \{ g(t) | \Omega(t) \} \\ \leq C + \mathbf{WE} \{ g(t) | \Omega(t) \} \\ + \sum_{i \in I} \sum_{j \in J} Q_{ij}(t) \mathbf{E} \{ a_{ij}(t) - b_{ij}(t) | \Omega(t) \}. \end{aligned} \quad (19)$$

By rearranging (19), we can obtain (11). ■

B. Online Algorithm

This paper aims to minimize the *drift plus cost*'s upper bound, which is shown in the R.H.S. of the inequation (11). An efficient DCOA is proposed which decomposes the optimization problem into a series of subproblems, and solves these subproblems concurrently in a distributed way.

In each slot t , DCOA dynamically makes the offloading decisions to optimize the R.H.S. of the inequation (11). In the optimization problem, since C and $b_{ij}(t)$ are constant, and the offloading decisions among different applications are independent, we decompose the central optimization problem to solving (20)–(21) for each application i concurrently

$$\min_{a_{ij}(t)} \sum_{j \in J} Q_{ij}(t) a_{ij}(t) + W \left[\varphi_i \left(\sum_{j=1}^n a_{ij}(t), t \right) + \phi_i(a_{i(n+1)}(t), t) \right] \quad (20)$$

subject to

$$\sum_{j \in J} a_{ij}(t) = A_i(t). \quad (21)$$

In slot t , let $e_i(t) = \sum_{j=1}^n a_{ij}(t)$ denote the number of tasks offloaded to the MEC. Thus, it holds that

$$a_{i(n+1)}(t) = A_i(t) - e_i(t). \quad (22)$$

Then, problem (20)–(21) can be transformed to problem (23)–(25)

$$\begin{aligned} \min_{e_i(t), a_{ij}(t)} \quad & \sum_{j=1}^n Q_{ij}(t) a_{ij}(t) + Q_{i(n+1)}(t) (A_i(t) - e_i(t)) \\ & + W[\varphi_i(e_i(t), t) + \phi_i(A_i(t) - e_i(t), t)] \end{aligned} \quad (23)$$

subject to

$$0 \leq e_i(t) \leq A_i(t) \quad (24)$$

$$e_i(t) = \sum_{j=1}^n a_{ij}(t). \quad (25)$$

To solve the minimization problem, we need determine the variables of $e_i(t)$ and $a_{ij}(t)$. Let us consider a simple case to solve this minimization problem, where the value of $e_i(t)$ has been given and known in advance. Then, the minimization problem (23)–(25) can be equivalent to

$$\min_{a_{ij}(t)} \sum_{j=1}^n Q_{ij}(t) a_{ij}(t) \quad (26)$$

subject to

$$\sum_{j=1}^n a_{ij}(t) = e_i(t). \quad (27)$$

We can regard problem (26) as one min-weight problem, and the weight for the number of offloaded tasks $a_{ij}(t)$ is $Q_{ij}(t)$. Therefore, the optimal solution can be obtained according to

$$a_{ij}(t) = \begin{cases} e_i(t), & j = j^* \\ 0, & \text{otherwise} \end{cases} \quad (28)$$

where $j^* \in \arg \min_{j \in \{1, 2, \dots, n\}} Q_{ij}(t)$.

Thus, for a given $e_i(t)$, we can obtain the optimal offloading decisions $a_{ij}(t)$ according to (22) and (28). Note that the decision of $e_i(t)$ has not been made yet. Based on the discussions above, we can rewrite $\sum_{j=1}^n Q_{ij}(t) a_{ij}(t)$ as $Q_{ij^*}(t) e_i(t)$. Then, the task offloading problem can be solved as

$$\min_{e_i(t)} Q_{ij^*}(t) e_i(t) + W[\varphi_i(e_i(t), t) + \phi_i(A_i(t) - e_i(t), t)] \quad (29)$$

subject to constraint (24).

Problem (29) is an integer-programming problem, where $e_i(t)$ is the only decision variable. The optimal solution depends on the concrete functions of $\varphi_i(\cdot)$ and $\phi_i(\cdot)$. After the variable $e_i(t)$ is determined, the offloading decisions $a_{ij}(t)$ can be made according to (22) and (28), respectively. Then, DCOA updates the queue length $Q_{ij}(t)$ according to (5).

We show the detailed algorithm in Algorithm 1.

Algorithm 1 DCOA

```

1: Obtain the queue length  $Q_{ij}(t)$ .
2: for all the application  $i$  do
3:   Determine the variable  $e_i(t)$  by solving the integer
     programming problem (29).
4:   for all the server  $j \in \{1, 2, \dots, n\}$  do
5:     Get the index  $j^*$  with the minimum value of  $Q_{ij}(t)$ ,
       i.e.,  $j^* \in \operatorname{argmin}_{j \in \{1, 2, \dots, n\}} Q_{ij}(t)$ .
6:     Set  $a_{ij}(t)$  according to (28).
7:   end for
8:   Set  $a_{i(n+1)}(t)$  according to (22).
9: end for

```

IV. ALGORITHM ANALYSIS

It is proven that by changing the value of W , DCOA can arbitrarily approach the long-term time average optimal offloading cost, and bound the queue length at the same time. The time average queue backlog during a long time is defined as follows:

$$\bar{Q} = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i \in I} \sum_{j \in J} \mathbf{E}\{Q_{ij}(t)\}. \quad (30)$$

Next, we present Lemma 1 which will be useful for analyzing the DCOA.

Lemma 1: For the system, let Λ denote its capacity region. Then, no matter what the possible value of offloadable task arrival rate $\lambda \in \Lambda$ is, if $A_i(t)$ is independent and identically distributed over slots, then, a randomized offloading strategy π^* can obtain the following results:

$$\begin{aligned} \mathbf{E}\{g^{\pi^*}(t)\} &= g^*(\lambda) \\ \mathbf{E}\left\{\sum_{j \in J} a_{ij}^{\pi^*}(t)\right\} &\leq \mathbf{E}\left\{\sum_{j \in J} b_{ij}(t)\right\}. \end{aligned}$$

Specifically, $g^*(\lambda)$ represents the optimal offloading cost with the offloadable task arrival rate λ .

Proof: Based on the Caratheodory's theorem [10], Lemma 1 can be derived. The details for proof are omitted here for ease of reading. ■

Recall that $A_i(t)$ is upper bounded by $A_i^{\max}$, and thus the objective function g is both upper bounded and lower bounded. Let $\hat{g}$ and $\check{g}$ define the upper bound and lower bound of g , respectively. Then, based on Lemma 1, we propose Theorem 2.

Theorem 2: If $\exists \varepsilon > 0$ satisfies $\lambda + \varepsilon \in \Lambda$, then, with our online algorithm, no matter what the possible value of W is, there exists upper bound for $\bar{Q}$ expressed in

$$\bar{Q} \leq \frac{C + W(\hat{g} - \check{g})}{\varepsilon}. \quad (31)$$

Furthermore, the long-term time average offloading cost can be bounded by (32). Here, C is the constant defined in Theorem 1. This shows that the offloading cost of our online algorithm can approximate the minimal cost by increasing W

$$g^{\text{DCOA}} \leq g^* + \frac{C}{W}. \quad (32)$$

Proof: Based on Lemma 1, since it holds that $\lambda + \varepsilon \in \Lambda$, there exists the randomized offloading strategy π' that satisfies the following results exists:

$$\mathbf{E}\{g^{\pi'}(t)\} = g^*(\lambda + \varepsilon) \quad (33)$$

$$\mathbf{E}\left\{\sum_{j \in J} a_{ij}^{\pi'}(t)\right\} \leq \mathbf{E}\left\{\sum_{j \in J} b_{ij}(t)\right\} - \varepsilon. \quad (34)$$

Recall that for all the possible policies (including π'), DCOA optimizes the R.H.S of the inequation (11). Therefore, (35) holds

$$\begin{aligned} \Delta(\Omega(t)) + W\mathbf{E}\{g(t)|\Omega(t)\} \\ \leq C + W\mathbf{E}\{g^{\pi'}(t)|\Omega(t)\} \\ + \sum_{i \in I} \sum_{j \in J} Q_{ij}(t) \mathbf{E}\{a_{ij}^{\pi'}(t) - b_{ij}(t)|\Omega(t)\}. \end{aligned} \quad (35)$$

Applying (33) and (34) to (35), and making use of iterated expectations, (36) holds

$$\begin{aligned} \mathbf{E}\{H(\Omega(t+1)) - H(\Omega(t))\} + W\mathbf{E}\{g(t)\} \\ \leq C + Wg^*(\lambda + \varepsilon) - \varepsilon \sum_{i \in I} \sum_{j \in J} \mathbf{E}\{Q_{ij}(t)\}. \end{aligned} \quad (36)$$

Subtracting $W\mathbf{E}\{g(t)\}$ on both sides of (36), it holds

$$\begin{aligned} \mathbf{E}\{H(\Omega(t+1)) - H(\Omega(t))\} \\ \leq C + W(g^*(\lambda + \varepsilon) - \mathbf{E}\{g(t)\}) - \varepsilon \sum_{i \in I} \sum_{j \in J} \mathbf{E}\{Q_{ij}(t)\} \\ \leq C + W(\hat{g} - \check{g}) - \varepsilon \sum_{i \in I} \sum_{j \in J} \mathbf{E}\{Q_{ij}(t)\}. \end{aligned} \quad (37)$$

We consider a general case that all the queues are empty at the beginning. By summing (37) over the time slots t , together with $H(\Omega(t)) \geq 0$, it yields

$$\begin{aligned} \varepsilon \sum_{t=0}^{T-1} \sum_{i \in I} \sum_{j \in J} \mathbf{E}\{Q_{ij}(t)\} &\leq (C + W(\hat{g} - \check{g}))T - \mathbf{E}\{H(\Omega(T))\} \\ &\leq (C + W(\hat{g} - \check{g}))T. \end{aligned} \quad (38)$$

Then, if we use εT to divide (38), and take a lim as $T \rightarrow \infty$, we can obtain (31).

Summing both sides of (36) over the time slots, and combining that $\forall i \in I, j \in J, \mathbf{E}\{Q_{ij}(t)\} \geq 0$, it holds

$$W \sum_{t=0}^{T-1} \mathbf{E}\{g(t)\} \leq (Wg^*(\lambda + \varepsilon) + C)T. \quad (39)$$

If we divide (39) by WT , we can yield

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbf{E}\{g(t)\} \leq g^*(\lambda + \varepsilon) + \frac{C}{W}. \quad (40)$$

Then, taking a lim of (40) as $T \rightarrow \infty$, based on the Lebesgue's dominated convergence theorem, and making $\varepsilon \rightarrow 0$, (32) can be obtained. ■

Through Theorem 2, we can see that the queue length is upper bounded by $O(W)$ and offloading cost is upper bounded by $O(1/W)$. When W increases, the queue length

would increase and offloading cost would decrease. And in the experiments, we can also see from Figs. 1 and 2 that with the increase of W , offloading cost decreases and queue length rises. It shows that DCOA can achieve a $[O(W), O(1/W)]$ tradeoff between the queue length and offloading cost. With the increase of W , it can approximate the minimum cost. Although the queue length rises with the increase of W , it can still be upper bounded. We can see that by letting ζ take the value of $[(C + W(\hat{g} - \check{g}))/\varepsilon]$, constraint (6) is satisfied. In practice, the parameter W can be decided by application providers or users according to actual demands.

V. EXPERIMENTS

In this section, both parameter analysis and comparison experiments are conducted. We evaluate the performance of DCOA using the machine with a quad core Intel CPU at 3.3 GHz frequency. In the experiments, we consider that there are three types of edge servers in the MEC system, and each type is with two edge servers providing computing services. There are two types of applications running in the IoT devices, and the arrival process of offloaded tasks for each application follows Poisson distribution [14] with rate of $\lambda_i = 160$. Note that DCOA can deal with the case with no statistical information of task arrivals. The service capacities of the MEC and back-end cloud are set as $\alpha_i \cdot \lambda_i$ and $\beta_i \cdot \lambda_i$, respectively, where $\alpha_i < 1$ and $\beta_i > 1$. As far as we know, there is no standard experimental platform or test data sets for the offloading cost. Thus, we use synthetic data for offloading cost in the experiments. According to [12], we set the cost function of offloading tasks to the MEC as $\phi_i(x, t) = \gamma_i(t) \cdot x$, where $\gamma_i(t)$ represents the unit offloading cost from application i to the MEC. The cost function of offloading tasks to back-end cloud is set as $\phi_i(x, t) = \varrho \gamma_i(t) \cdot x$ where $\varrho > 1$.

A. Parameter Analysis

1) *Impact of Tradeoff Parameter*: Figs. 1 and 2 show the time-average offloading cost and queue length of DCOA with different values of W . We can see in Fig. 1 that the offloading cost decreases with the increase of W , which agrees with (32) in Theorem 2. The reason is that as the value of W increases, DCOA puts more weight on the offloading cost and tries to reduce the offloading cost. However, Fig. 2 shows that with increase of W , the queue length also rises, which agrees with (31) in Theorem 2. Nevertheless, with far more increase of W , the queue length would stabilize gradually. It illustrates that the queue length of DCOA can be upper bounded. Together with Figs. 1 and 2, we can see that our DCOA can make a tradeoff between the offloading cost and queue length by changing the value of W .

Figs. 3 and 4 show the number of task requests offloaded to the MEC and the back-end cloud with different values of W . We can see from Fig. 3 that the number of tasks offloaded to the MEC rises with the increase of W . This is because a larger value of W means more weight put on the offloading cost, and DCOA would offload more tasks to the MEC to reduce the overall offloading cost. Correspondingly, Fig. 4 shows that

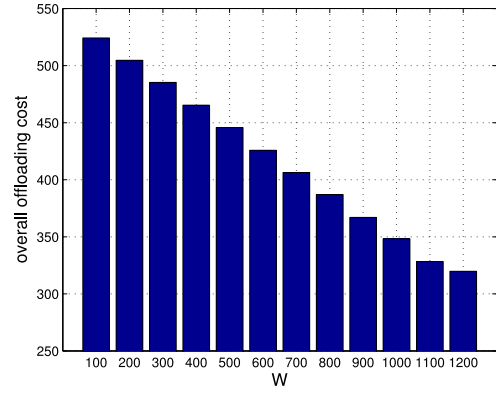


Fig. 1. Overall offloading cost with different values of W .

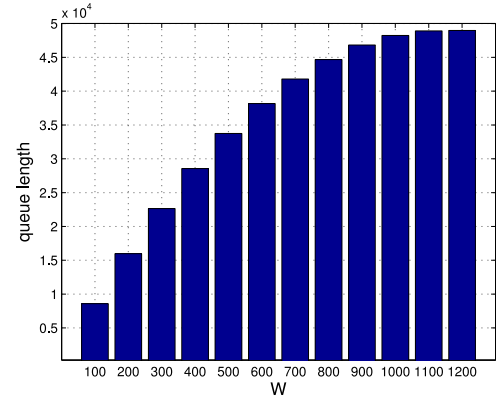


Fig. 2. Queue length with different values of W .

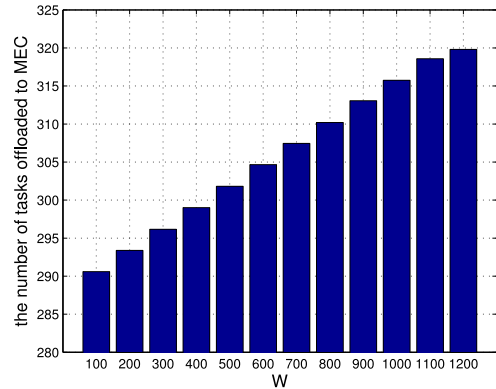


Fig. 3. Number of tasks offloaded to MEC with different values of W .

with the increase of W , the number of tasks scheduled to the back-end cloud would decrease.

2) *Impact of Offloaded Task Arrival Rate*: To validate the DCOA's performance with different arrival rates of offloaded tasks, the arrival rate of offloaded tasks for i -th application is set as $p \cdot \lambda_i$. In the experiment, we consider three different settings, where $p = 0.8, 1$, and 1.2 , respectively.

Figs. 5 and 6 show the overall offloading cost and queue length with different arrival rates of offloaded tasks. As the offloaded task arrival rate increases, both the offloading cost and queue length would rise. In addition, from Figs. 7 and 8, we can see the number of tasks offloaded to the MEC and back-end cloud with different arrival rates of offloaded tasks.

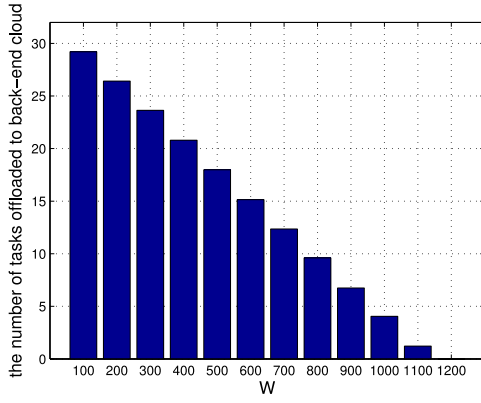


Fig. 4. Number of tasks offloaded to back-end cloud with different values of W .

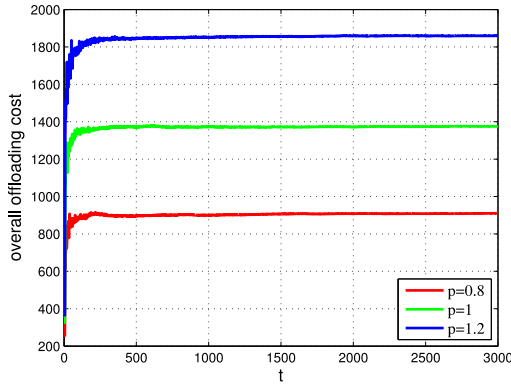


Fig. 5. Overall offloading cost with different arrival rates.

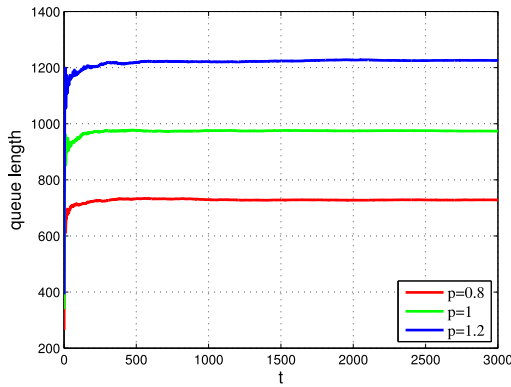


Fig. 6. Queue length with different arrival rates.

Similar phenomenon can be observed that with the increase of task arrival rates, the number of tasks offloaded to the MEC would increase, and the number of tasks offloaded to the back-end cloud also rises. It can also be observed that the offloading cost, queue length, number of tasks offloaded to the MEC and back-end cloud all stabilize quickly under different arrival rates. This shows that DCOA can make the offloading decisions dynamically to adopt to different arrival rates, and maintain the system stability quickly.

B. Comparison Experiment

We further evaluate DCOA's performance by comparing with two other algorithms: 1) randomized algorithm which

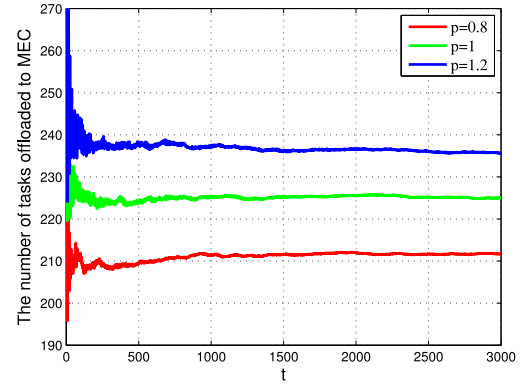


Fig. 7. Number of tasks offloaded to MEC with different arrival rates.

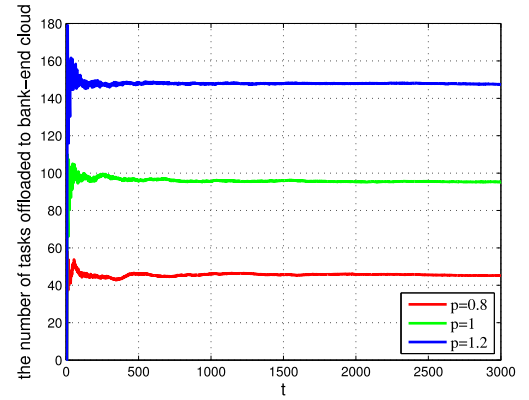


Fig. 8. Number of tasks offloaded to back-end cloud with different arrival rates.

offloads the task requests to the MEC or back-end cloud randomly and 2) MEC Only algorithm which offloads all the task requests to the MEC for processing.

Figs. 9 and 10 show the overall offloading cost and queue length of three different algorithms. Fig. 9 shows that the offloading cost of MEC Only algorithm is the lowest, since it offloads all the tasks to the MEC with lower offloading cost. The offloading cost of our DCOA is much smaller than the Randomized algorithm. We can see from Fig. 10 that the queue length of the MEC only algorithm increases continuously along with the time, resulting in system instability. DCOA's queue length is similar to that of the randomized algorithm, and maintains at a low level. This shows that our DCOA can effectively reduce the queue length and keep the overall system in a low congestion state. Combining Figs. 9 and 10, we can see that our DCOA can reduce the offloading cost effectively while maintaining the low congestion state of system.

We also conduct experiments to investigate how long it takes to reach the tradeoff between offloading cost and performance. We conduct the experiments for 200 runs, and then, average the results. In Figs. 11 and 12, we plot the offloading cost and queue length of the three different algorithms in the first 200 time slots. It can be seen from Fig. 11 that the offloading cost of DCOA would stabilize after the first 100 time slots. And Fig. 12 shows that the queue length of DCOA would become stable after the first 50 time slots. Together with Figs. 11 and 12, we can see that it takes about

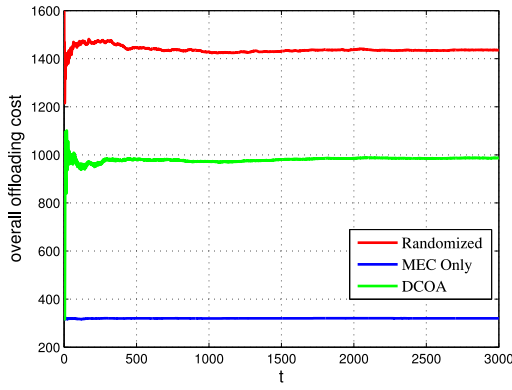


Fig. 9. Overall offloading cost of different algorithms.

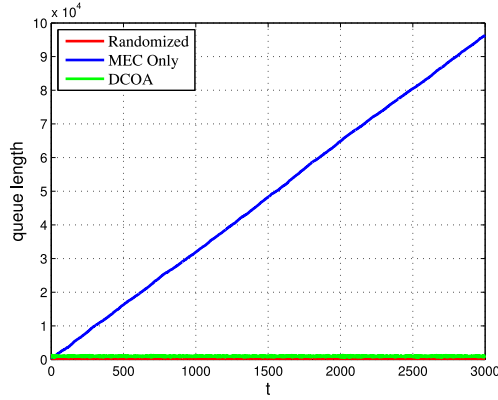


Fig. 10. Queue length of different algorithms.

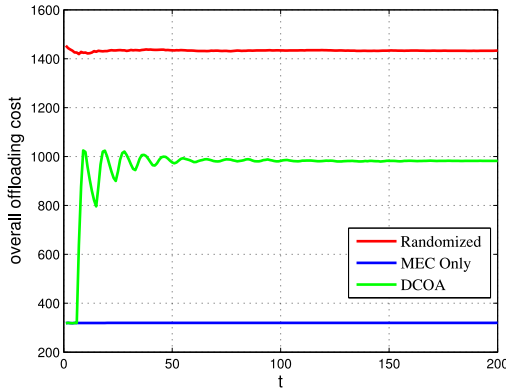


Fig. 11. Overall offloading cost of different algorithms in first 200 time slots.

100 time slots for DCOA to reach the tradeoff between the offloading cost and performance. the randomized algorithm can also achieve the tradeoff between the offloading cost and performance quickly. However, the MEC Only algorithm is unstable.

VI. RELATED WORK

As the applications running on terminal devices are becoming complex and computation-intensive, offloading tasks of computation-intensive applications from terminal devices to remote cloud data centers has been a hot research topic and drawn extensive attention. Chun *et al.* [15] designed

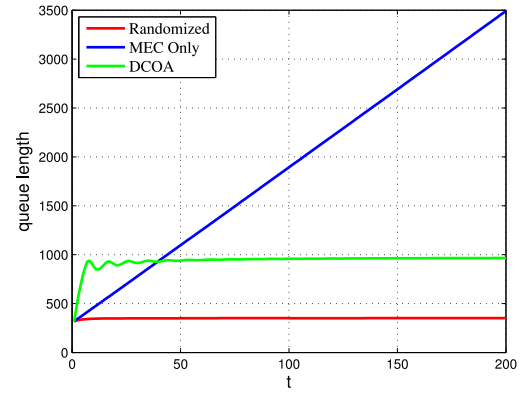


Fig. 12. Queue length of different algorithms in first 200 time slots.

and implemented CloneCloud to offload tasks from terminal devices to device clones with VM images in computational cloud. Kosta *et al.* [16] proposed ThinkAir, a framework that took advantage of parallelizing method execution to offload device applications to the cloud. Gordon *et al.* [17] proposed the COMET system which took advantage of the migration of threads by making use of the underlying memory model to implement distributed shared memory. Zhang *et al.* [18] focused on mobile cloud computing system, and conducted research on task scheduling to save energy for the terminal device. For offloading in mobile cloud systems, Kwak *et al.* [19] studied dynamic resource and task allocation problem, and proposed a DREAM algorithm to save energy.

However, with the promotion of IoT, the number of devices is rising dramatically, and there is a large quantity of data generated. Thus, it will lead to a heavy load on the network to offload all the application data and tasks to the cloud [6]. Therefore, there is an increasing trend to place servers at the network's edge. Several technologies, like edge computing, fog computing, cloudlet, femto cloud have been proposed. Service migration and task scheduling among multiple edge computing systems were studied in [12], and both service migration and task offloading policies were proposed. However, they assumed that each edge-cloud can host at most one application. Focusing on one wireless metropolitan area network, Jia *et al.* [9] conducted research on the placement of cloudlet and allocation of user to the cloudlets. They assumed the offloadable tasks arrived at the system according to the Poisson process. However, in reality, the task arrival process is highly dynamic, and the statistical information of task arrival can hardly be obtained or precisely predicted [20], [21].

The application offloading in a femto cloud combining radio and computational resources was studied to minimize energy in [22]. Yu *et al.* [23] considered an edge computing system, where multiple terminal devices can access by orthogonal frequency-division multiplexing access (OFDMA), and studied the subcarrier allocation for task offloading. For the edge computing system with two different access modes, including time-division multiple access and OFDMA, You *et al.* [24] focused on the resource allocation problem and formulated the allocation as a convex optimization problem and a mixed-integer problem, respectively. Most of these works were

based on the assumption that after the old tasks were finished or dropped, new tasks could be generated. And they focused mainly on short-term performance, where the associated optimization problems were typically deterministic. In this paper, we considered stochastic models and focused on long-term performance over a large time horizon.

Kim *et al.* [25] studied the cost-delay tradeoff in an edge computing system and proposed an algorithm for dual-side optimization. Liu *et al.* [26] studied the task offloading in the edge computing system. They adopted a Markov decision process approach, and formulated a problem aiming to minimize delay with power constraints. Then, they accomplished offloading based on one searching algorithm. Lyu *et al.* [27] combined both throughput and fairness, and generated asymptotically optimal schedules for offloading of MEC for IoT. Most of these works only considered the task scheduling within the edge computing systems. However, for the edge computing system, its resource and computation capacity are still limited. Therefore, it is not a good solution to offload all the tasks to the edge computing system. The reason is that too many tasks complete for the limited resource, then the system will tend to be unstable [8]. In this paper, we study the task scheduling problem in a cooperative way combining the edge computing system and the back-end cloud.

VII. CONCLUSION

This paper studies the dynamic computation offloading problem in edge computing for IoT. An optimization model is formulated to address the research problem, where the objective is to minimize the offloading cost while providing performance guarantees. To solve this optimization problem, we design an online and distributed algorithm called DCOA, which makes dynamic offloading decisions to adapt to external environment changes. DCOA decomposes the original offloading problem to a series of subproblems, and solves each one efficiently in a parallel way. We provide mathematical analysis to demonstrate the effectiveness of DCOA. Experiments are also conducted which further validate DCOA's effectiveness.

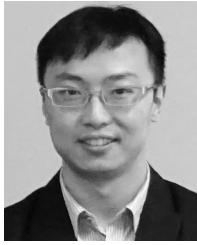
REFERENCES

- [1] J. Lin *et al.*, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.
- [2] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for smart cities," *IEEE Internet Things J.*, vol. 1, no. 1, pp. 22–32, Feb. 2014.
- [3] W. Quan, Y. Liu, H. Zhang, and S. Yu, "Enhancing crowd collaborations for software defined vehicular networks," *IEEE Commun. Mag.*, vol. 55, no. 8, pp. 80–86, Aug. 2017.
- [4] Q. Ye and W. Zhuang, "Token-based adaptive MAC for a two-hop Internet-of-Things enabled MANET," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1739–1753, Oct. 2017.
- [5] "Cisco global cloud index: Forecast and methodology, 2014–2019," San Jose, CA, USA, Cisco, White Paper, 2014.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [7] J. Ren, H. Guo, C. Xu, and Y. Zhang, "Serving at the edge: A scalable IoT architecture based on transparent computing," *IEEE Netw.*, vol. 31, no. 5, pp. 96–105, 2017.
- [8] H. Tan, Z. Han, X.-Y. Li, and F. C. M. Lau, "Online job dispatching and scheduling in edge-clouds," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, May 2017, pp. 1–9.
- [9] M. Jia, J. Cao, and W. Liang, "Optimal cloudlet placement and user to cloudlet allocation in wireless metropolitan area networks," *IEEE Trans. Cloud Comput.*, vol. 5, no. 4, pp. 725–737, Oct./Dec. 2017.
- [10] M. J. Neely, *Stochastic Network Optimization With Application to Communication and Queueing Systems*. San Rafael, CA, USA: Morgan & Claypool, 2010.
- [11] F. Liu *et al.*, "On arbitrating the power-performance tradeoff in SAAS clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 10, pp. 2648–2658, Oct. 2014.
- [12] R. Ugaonkar *et al.*, "Dynamic service migration and workload scheduling in edge-clouds," *Perform. Eval.*, vol. 91, pp. 205–228, Sep. 2015.
- [13] D. Zhang, R. Shen, J. Ren, and Y. Zhang, "Delay-optimal proactive service framework for block-stream as a service," *IEEE Wireless Commun. Lett.*, vol. 7, no. 4, pp. 598–601, Aug. 2018.
- [14] E. Chlebus and J. Brazier, "Nonstationary Poisson modeling of Web browsing session arrivals," *Inf. Process. Lett.*, vol. 102, no. 5, pp. 187–190, 2007.
- [15] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "CloneCloud: Elastic execution between mobile device and cloud," in *Proc. 6th Conf. Comput. Syst.*, Salzburg, Austria, 2011, pp. 301–314.
- [16] S. Kosta, A. Aucinas, P. Hui, R. Mortier, and X. Zhang, "ThinkAir: Dynamic resource allocation and parallel execution in the cloud for mobile code offloading," in *Proc. IEEE INFOCOM*, Mar. 2012, pp. 945–953.
- [17] M. S. Gordon, D. A. Jamshidi, S. Mahlke, Z. M. Mao, and X. Chen, "COMET: Code offload by migrating execution transparently," in *Proc. 10th USENIX Conf. Oper. Syst. Design Implement.*, Los Angeles, CA, USA, 2012, pp. 93–106.
- [18] W. Zhang *et al.*, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Sep. 2013.
- [19] J. Kwak, Y. Kim, J. Lee, and S. Chong, "DREAM: Dynamic resource and task allocation for energy minimization in mobile cloud systems," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 12, pp. 2510–2523, Dec. 2015.
- [20] D. Zhang *et al.*, "Resource allocation for green cloud radio access networks with hybrid energy supplies," *IEEE Trans. Veh. Technol.*, vol. 67, no. 2, pp. 1684–1697, Feb. 2018.
- [21] D. Zhang *et al.*, "Utility-optimal resource management and allocation algorithm for energy harvesting cognitive radio sensor networks," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3552–3565, Dec. 2016.
- [22] O. Munoz, A. Pascual-Iserte, and J. Vidal, "Optimization of radio and computational resources for energy efficiency in latency-constrained application offloading," *IEEE Trans. Veh. Technol.*, vol. 64, no. 10, pp. 4738–4755, Oct. 2015.
- [23] Y. Yu, J. Zhang, and K. B. Letaief, "Joint subcarrier and CPU time allocation for mobile edge computing," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, Dec. 2016, pp. 1–6.
- [24] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017.
- [25] Y. Kim, J. Kwak, and S. Chong, "Dual-side optimization for cost-delay tradeoff in mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 2, pp. 1765–1781, Feb. 2018.
- [26] J. Liu, Y. Mao, J. Zhang, and K. B. Letaief, "Delay-optimal computation task scheduling for mobile-edge computing systems," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2016, pp. 1451–1455.
- [27] X. Lyu *et al.*, "Optimal schedule of mobile edge computing for Internet of Things using partial information," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2606–2615, Nov. 2017.



Ying Chen received the B.Eng. degree from the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China, in 2012, and the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, in 2017.

She is an Assistant Professor with Computer School, Beijing Information Science and Technology University, Beijing. Her current research interests include edge computing, cloud computing, services computing, performance evaluation, and optimization.



Ning Zhang (GS'12–M'14) received the Ph.D. degree from the University of Waterloo, Waterloo, ON, Canada, in 2015.

He is an Assistant Professor with Texas A&M University Corpus Christi, Corpus Christi, TX, USA. He was a Post-Doctoral Research Fellow with the University of Waterloo and the University of Toronto, Toronto, ON, Canada, respectively. His current research interests include next generation mobile networks, physical layer security, machine learning, and mobile edge computing.

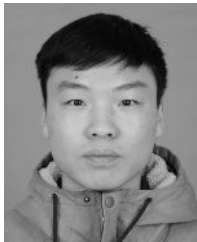
Dr. Zhang was a recipient of the Best Paper Award of IEEE Globecom 2014 and IEEE WCSP 2015. He is serves/served as an Associate Editor of IEEE *ACCESS* and *IET Communication*, an Area Editor of the *Encyclopedia of Wireless Networks* (Springer) and a Cambridge Scholars and a Guest Editor of *Wireless Communication and Mobile Computing*, the *International Journal of Distributed Sensor Networks*, and *Mobile Information System*. He also served as the Workshop Chair for the first IEEE Workshop on Cooperative Edge and IEEE Workshop on Mobile Edge Networks and Systems for Immersive Computing and IoT.



Xin Chen received the Ph.D. degree in computer science from the Beijing Institute of Technology, Beijing, China.

He is currently a Professor with the School of Computer Science, Beijing Information Science and Technology University, Beijing. He was a Post-Doctoral Fellowship of Computer Architecture from Tsinghua University, Beijing, in 2006. His current research interests include performance evaluation of wireless networks.

He is a Senior Member of the China Computer Federation (CCF), a member of the CCF Technical Committee of Theoretical Computer Science, and the CCF Technical Committee of Petri Nets.



Yongchao Zhang received the B.Eng. degree from the School of Information and Communication Engineering, Beijing Information Science and Technology University, Beijing, China, in 2017, where he is currently pursuing the Graduation degree at the School of Computer Science.

His current research interests include wireless networks and edge computing.